Тема 3. Анализ алгоритмов и их сложности

Лекция 5

Анализ алгоритмов и их сложности

Существует ряд важных причин для анализа алгоритмов. Одной из них является необходимость получения оценок или границ для объема памяти или времени работы, которое потребуется алгоритму для успешной обработки конкретных данных.

Для оценки качества алгоритма вводится понятие *сложность алгоритма,* или обратное понятие — *эффективность алгоритма.* Чем большее время и объем памяти требуются для реализации алгоритма, тем больше его сложность и соответственно ниже эффективность. Сложность алгоритма делится на временную и емкостную. Временная сложность - это критерий, характеризующий временные затраты на реализацию алгоритма. Емкостная сложность - критерий, характеризующий затраты памяти на те же цели. В зависимости от конкретной формы этих критериев сложность алгоритма в свою очередь подразделяется на *практическую* и *теоретическую.* Практическая временная сложность обычно оценивается во временных единицах (секунды, милисекунды, количество временных тактов процессора, количество выполнения циклов и т.п.). Практическая емкостная сложность выражается, как правило, в битах, байтах, словах и т.п. Способы представления теоретической сложности алгоритма будут рассмотрены далее.

Перечислим основные факторы, от которых может зависеть сложность алгоритма:

- быстродействие компьютера и его емкостные ресурсы (в первую очередь - объем оперативной памяти). В самом деле, чем ниже тактовая частота процессора и меньше объем оперативного запоминающего устройства, тем медленнее выполняются

арифметические и логические операции, тем чаще (для больших задач) приходится обращаться к медленно действующей внешней памяти, и, следовательно, больше времени уходит на реализацию алгоритма;

- выбранный язык программирования. Задача, запрограммированная, например, на языке Ассемблера, в общем случае решится быстрее, чем по тому же самому алгоритму, но запрограммированному на языке более высокого уровня, например на Паскале;
- выбранный математический метод формулирования задачи;
- искусство и опыт программиста. В общем случае по одному и тому же алгоритму опытный программист напишет более эффективно работающую программу, чем его начинающий коллега.

Для решения многих проблем существует множество различных алгоритмов. Какой из них выбрать для решения конкретной задачи? Этот вопрос очень тщательно прорабатывается в программировании. Эффективность программы является очень важной ее характеристикой. Эффективность программы имеет две составляющие: память (или пространство) и время. Пространственная эффективность измеряется количеством памяти, требуемой для выполнения программы.

Компьютеры обладают ограниченным объемом памяти. Если две программы реализуют идентичные функции, то та, которая использует меньший объем памяти, характеризуется большей пространственной эффективностью. Иногда память становится доминирующим фактором в оценке эффективности программ. Однако в последние годы в связи с быстрым ее удешевлением эта составляющая эффективности постепенно теряет свое значение. Временная эффективность программы определяется временем, необходимым для ее выполнения.

Лучший способ сравнения эффективностей алгоритмов состоит в сопоставлении их порядков сложности. Этот метод применим как к временной, так и пространственной сложности. Порядок сложности алгоритма выражает его эффективность обычно через количество обрабатываемых данных.

Целью данной темы является рассмотрение вопросов оценки временной сложности алгоритмов. В ней демонстрируется существование нескольких алгоритмов решения некоторых задач, и рассматривается проблема сравнения и выбора алгоритма из множества эквивалентных по решаемым задачам алгоритмов. Вводиться понятие временной сложности алгоритма, рассматривается математический аппарат для оценки временной сложности и правила применения этого аппарата.

3.1. Задачи и многообразие алгоритмов их решения

За многовековую практику человечество убедилось, что для большинства задач существует более одного способа их решения, и, следовательно, можно сформулировать несколько алгоритмов, приводящих к одному и тому же результату. В качестве примера, иллюстрирующего такое положение дел, рассмотрим задачу возведения в степень.

Задача возведения в степень. Дано число x и натуральное (целое неотрицательное) число $n \geq 0$. Вычислить значение функции $f(x) = x^n$.

Очевидный способ решения этой задачи следует из определения операции возведения в степень: искомый результат есть:

$$x^n = \underbrace{x \cdot x \cdot \ldots \cdot x}_{n\ раз} \tag{3.1}$$

Положим $f = 1$, далее в цикле, повторяющемся n раз, вычислим произведение $f=f*x$. По окончании цикла f будет содержать искомый результат $f(x) = x^n$. На рис. 3.1 приведена блок-схема этого алгоритма.

Однако можно предложить и другие способы решения. Проведем анализ формулы (3.1):

$$x^n = \underbrace{x \cdot x \cdot \ldots \cdot x}_{n\ раз} = \begin{cases} \underbrace{x \cdot x \cdot \ldots \cdot x}_{n \operatorname{div} 2\ раз} \cdot x \cdot \underbrace{x \cdot x \cdot \ldots \cdot x}_{n \operatorname{div} 2\ раз}, & если\ n - нечетное \\ \underbrace{x \cdot x \cdot \ldots \cdot x}_{n \operatorname{div} 2\ раз} \underbrace{x \cdot x \cdot \ldots \cdot x}_{n \operatorname{div} 2\ раз}, & если\ n - четное \end{cases} \tag{3.2}$$

Здесь и далее символ *div* обозначают операцию целочисленного деления.

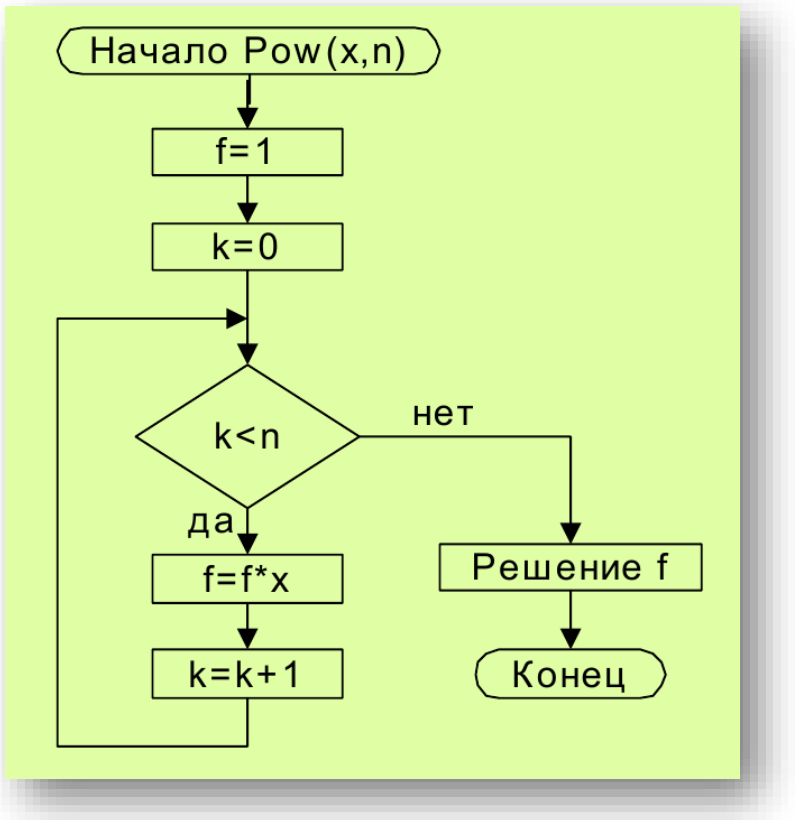


Рис. 3.1. Тривиальный алгоритм возведения в степень

Из формулы (3.2) следует, так называемое рекурсивное определение операции возведения в степень:

$$x^n = \begin{cases} 1, & \text{если } n = 0 \\ x \cdot \left(x^{n\,div\,2}\right)^2, & \text{если } n - \text{нечетное} \\ \left(x^{n\,div\,2}\right)^2, & \text{если } n - \text{четное} \end{cases} \qquad (3.3)$$

Определение (3.3) дает основания к разработке *рекурсивного алгоритма возведения в степень*. Его блок-схема представлена на рис. 3.2.

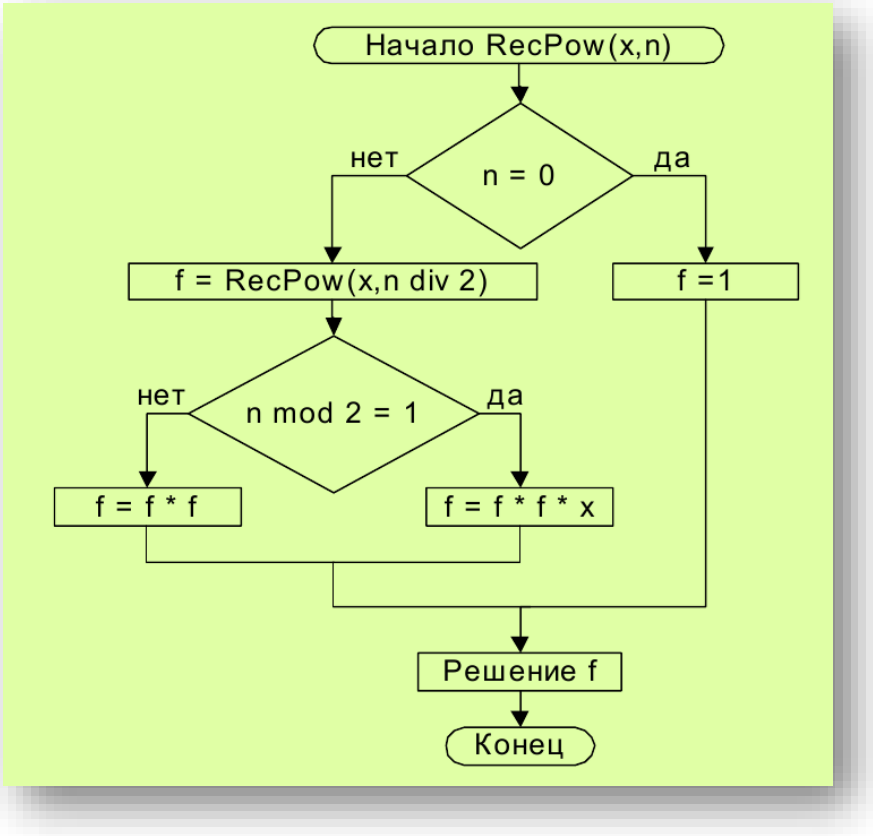


Рис. 3.2. Рекурсивный алгоритм возведения в степень

Рассмотрим еще один способ. Число n можно представить в виде:

$$n = d_m \cdot 2^m + d_{m-1} \cdot 2^{m-1} + ... + d_1 \cdot 2^1 + d_0 \cdot 2^0, \qquad (3.4)$$

где $d_i \in \{0,1\}$, $0 \le i \le m$ - двоичная цифра, $m = \lfloor log_2 n \rfloor$ – ближайшее целое не превосходящее $log_2 n$ (количество двоичных знаков, необходимых для записи числа минус 1). Тогда:

$$x^n = x^{d_m \cdot 2^m} \cdot x^{d_{m-1} \cdot 2^{m-1}} \cdot ... \cdot x^{d_1 \cdot 2^1} \cdot x^{d_0 \cdot 2^0}. \qquad (3.5)$$

Введем обозначения: $c_i = x^{2^i}$, $0 \le i \le m$, тогда формулу (3.5) можно представить в виде:

$$x^n = \prod_{i=0}^{m} c_i^{d_i}. \qquad (3.6)$$

Легко заметить, что $c_0 = x$, и $c_i = c_{i-1}^2$ для всех $1 \le i \le m$.

Другое важное замечание состоит в том, что:

$$c_i^{d_i} = \begin{cases} c_i, если\ d_i = 1 \\ 1, если\ d_i = 0 \end{cases}.$$

Учитывая формулу (3.6) и сделанные замечания можно предложить так называемый быстрый алгоритм возведения в степень. Его блок-схема представлена на рис. 3.3 а). Вообще, ставший классическим алгоритм быстрого возведения в степень выглядит так, как изображено на рис. 3.3 б).

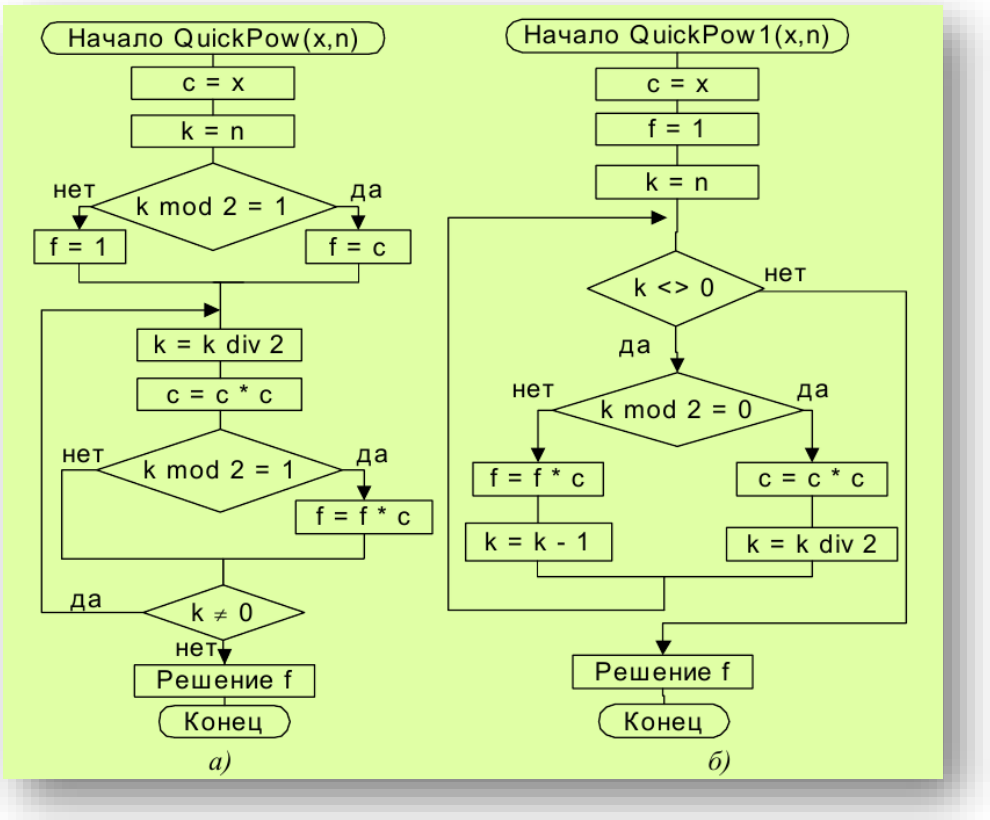


Рис. 3.3. Алгоритмы быстрого возведения в степень

И так, как мы смогли убедиться, для такой относительно простой задачи, как возведение в степень существуют, по крайней мере, 4 алгоритма.

3.2. Проблема выбора алгоритма. Понятие временной сложности

Если способов решения задачи может быть несколько, то становится вопрос выбора наилучшего из существующих алгоритмов. Что означает – наилучший алгоритм? Каков критерий этой наилучшести? Так что, выбор подходящего алгоритма вызывает определенные трудности. В самом деле, на чем основывать свой выбор, если алгоритм должен удовлетворять следующим противоречащим друг другу требованиям:

- быть простым для понимания, перевода в программный код и отладки;
- эффективно использовать компьютерные ресурсы и выполняться по возможности быстро.

Если написанная программа должна выполняться только несколько раз, то первое требование наиболее важно. Стоимость рабочего времени программиста обычно значительно превышает стоимость машинного времени выполнения программы, поэтому стоимость программы оптимизируется по стоимости написания (а не выполнения) программы.

Если мы имеем дело с задачей, решение которой требует значительных вычислительных затрат, то стоимость выполнения программы может превысить стоимость написания программы, особенно если программа должна выполняться многократно. Поэтому, с финансовой точки зрения, более предпочтительным может стать сложный комплексный алгоритм (в надежде, что результирующая программа будет выполняться существенно быстрее, чем более простая программа). Но и в этой ситуации разумнее сначала реализовать простой алгоритм, чтобы определить, как должна себя вести более сложная программа. При построении сложной программной системы желательно реализовать ее простой прототип, на котором можно провести необходимые измерения и смоделировать ее поведение в целом, прежде чем приступать к разработке окончательного варианта. Таким образом, программисты должны быть осведомлены не только о методах построения быстрых программ, но и знать, когда их следует применить (желательно с минимальными программистскими усилиями).

На время выполнения программы влияют следующие факторы:

1) ввод исходной информации в программу;
2) качество скомпилированного кода исполняемой программы;
3) машинные инструкции, используемые для выполнения программы;
4) временная сложность алгоритма соответствующей программы.

Поскольку время выполнения программы зависит от ввода исходных данных, его можно определить как функцию от исходных данных. Но зачастую время выполнения программы зависит не от самих исходных данных, а от их "размера". В общем случае длина (размерность) входных данных является наиболее подходящей мерой объема входной информации. Обычно говорят, что время выполнения программы имеет порядок $T(n)$ от входных данных размера n.

Однако для многих программ время выполнения действительно является функцией входных данных, а не их размера. В этой ситуации $T(n)$ определяется как время выполнения *в наихудшем случае*, т.е. как максимум времени выполнения по всем входным данным размера n, или время выполнения в наилучшем случае, т.е. как минимум выполнения по всем входным данным размера n.

Наиболее точной оценкой времени выполнения программы является аналитическое выражение для функции: $t=T(n)$, где t – время выполнения программы от момента ее запуска до получения окончательного результата. Для ее получения необходимо проделать множество экспериментов, запуская программу с различными исходными данными и в широких пределах варьируя их размерность. Результаты экспериментов необходимо затабулировать и подобрать функцию, наилучшем образом приближающую получившуюся таблицу.

Однако данный подход имеет множество недостатков. Во-первых, точное измерение времени выполнения программы невозможно. Во-вторых, функция, аппроксимирующая таблицу экспериментальных данных, существенно зависит от количества и качества проведенных экспериментов и, как правило, весьма приближенно отражает искомую временную зависимость. В третьих, на время выполнения существенное влияние

оказывают второй и третий из выше перечисленных факторов – компилятор, используемый для компиляции программы, и машина, на которой выполняется программа. При изменении компилятора и (или) компьютера, с помощью которых создается и исполняется программа, результаты измерения времени могут существенно отличаться.

Эти факторы влияют на то, что для измерения времени выполнения *T(n)* невозможно применить стандартные единицы измерения, такие как секунды или миллисекунды. Поэтому остается только делать заключения, подобные "время выполнения такого-то программы пропорционально n^2". Константы пропорциональности также нельзя точно определить, поскольку они зависят от компилятора, компьютера и других факторов.

Указанные соображения позволяют сделать вывод о том, что функциональная зависимость и единица измерения *T(n)* не может быть точно определена. В практике оценки времени выполнения программ, *T(n)* понимают как количество элементарных шагов – инструкций, выполняемых на идеализированном компьютере. Это позволяет абстрагироваться от факторов, вносимых конкретным компилятором и конкретной вычислительной машиной. Единственным значимым и определяющем фактором, в этом случае остается временная сложность алгоритма. Поэтому очень часто время выполнения программы отождествляют с временной сложностью ее алгоритма.

Таким образом, ***временной сложностью алгоритма*** называется зависимость времени выполнения алгоритма от количества обрабатываемых входных данных, измеряемое в "шагах" (инструкциях алгоритма), которые необходимо выполнить алгоритму для достижения запланированного результата.

3.3. Асимптотические соотношения оценки временной сложности

Как было показано ранее, время выполнения программы невозможно определить точно путем проведения серии экспериментов. Другой способ оценки временной сложности – математически оценить время исполнения подсчетом операций. Проиллюстрируем этот подход на примере задачи вычисления значения многочлена.

Задача вычисления значения многочлена. Дан многочлен степени *n*

$$P_n(x) = a_n \cdot x^n + a_{n-1} \cdot x^{n-1} + \ldots + a_i \cdot x^i + \ldots + a_1 \cdot x^1 + a_0, \qquad (3.7)$$

заданный массивом своих коэффициентов A={A[0], A[1],…, A[*n*]} и значением переменной *x*. Вычислить значение многочлена $S = P_n(x)$.

Можно предложить следующий способ ее решения (*алгоритм 1*): для каждого слагаемого, кроме a_0 возвести *x* в заданную степень последовательным умножением и затем помножить на коэффициент. Затем слагаемые сложить. На рис. 3.4 приведена блок-схема этого алгоритма.

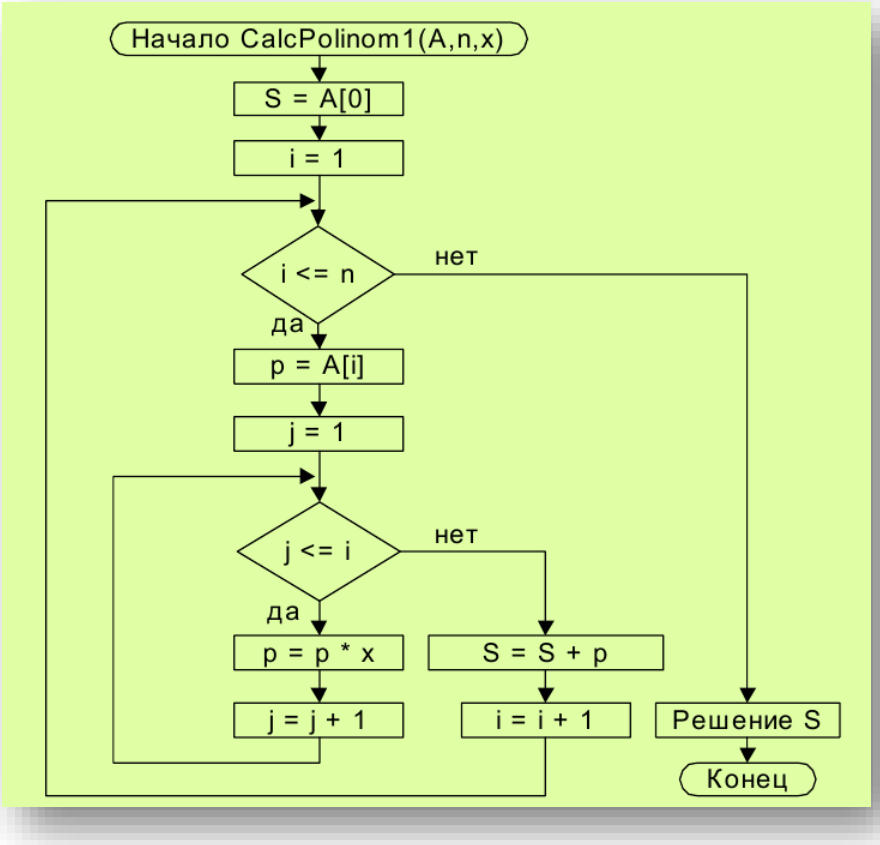


Рис. 3.4. Вычисления значения многочлена (алгоритм 1)

Для простоты анализа временной сложности *T(n)* будем принимать в расчет только операции, связанные с вычислением значения многочлена игнорируя операции сравнения, инициализации и изменения значений

параметров цикла. Вычисление *i*-го слагаемого(*i=1...n*) требует *i* умножений. Значит, всего $1 + 2 + 3 + ... + n = n(n+1)/2$ умножений. Кроме того, требуется *n* сложений и одна операция начального присваивания значения a_0. Таким образом, временная сложность этого алгоритма:

$T(n) = n(n+1)/2 + n + 1 = n^2/2 + 3n/2 + 1$ операций.

Теперь рассмотрим другой подход (*алгоритм 2*): в формуле (3.7) вынесем *x*-ы за скобки и перепишем многочлен в виде

$$P_n(x) = a_0 + x(a_1 + x(a_2 + ...(a_i + x(a_{n-1} + a_n \cdot x))...), \qquad (3.8)$$

Расстановка скобок диктует такой порядок вычислений:

$$S_0 = a_n,$$

$$S_1 = S_0 x + a_{n-1},$$

$$S_2 = S_1 x + a_{n-2},$$

$$...,$$

$$S_i = S_{i-1} x + a_{n-i},$$

$$...,$$

$$S_n = S_{n-1} x + a_0, P_n(x) = S_n$$

Рассмотренный метод называется схемой Горнера. На рис. 3.5 приведена блок-схема алгоритма вычисления значения многочлена по схеме Горнера.

Подсчет временной сложности в этом случае гораздо проще: для вычисления S_i требуется 1 умножение и 1 сложение. Всего такая итерация осуществляется *n* раз. Таким образом, временная сложность этого алгоритма: $T(n) = n$ умножений + n сложений = $2n$ операций.

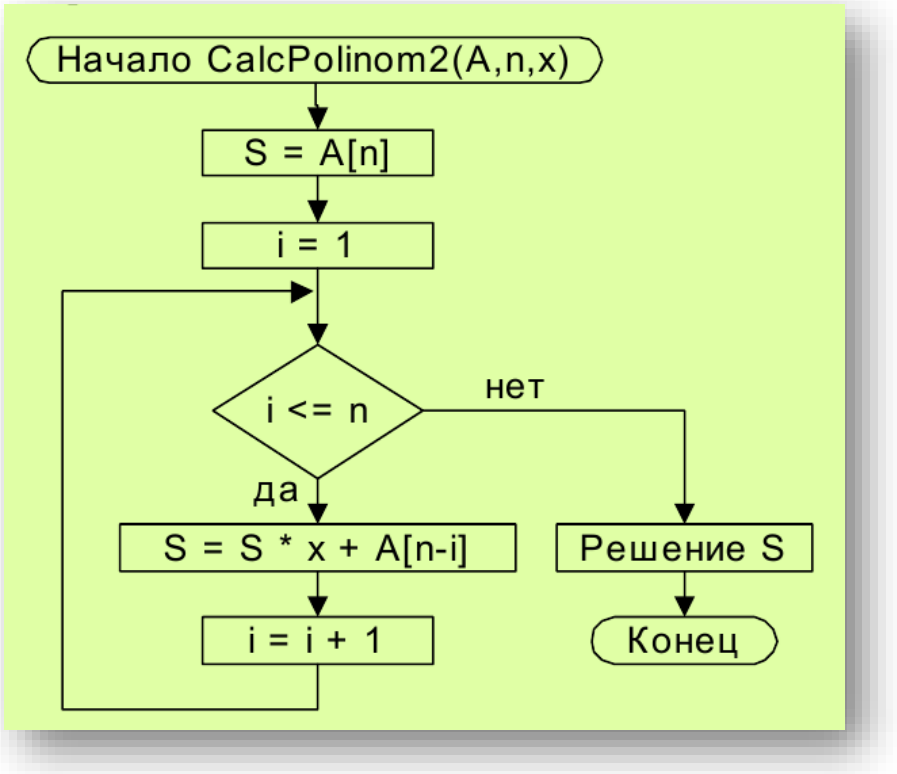


Рис. 3.5. Вычисления значения многочлена по схеме Горнера (алгоритм 2)

Зачастую такая подробная оценка временной сложности не требуется. Вместо нее приводят лишь асимптотическую скорость возрастания количества операций при увеличении *n*.

Так, например, функция $T(n) = n^2/2 + 3n/2 + 1$ возрастает приблизительно как $n^2/2$ (отбрасываем сравнительно медленно растущее слагаемое (3n/2+1). Константный множитель 1/2 также убираем и получаем асимптотическую оценку для алгоритма 1, которая обозначается специальным символом $O(n^2)$ (читается как "О большое от эн квадрат"). Это – верхняя оценка, т.е количество операций (а значит, и время работы) растет не быстрее, чем квадрат количества элементов. В этом случае говорят, что *T(n) есть* $O(n^2)$, или *T(n) имеет порядок* $O(n^2)$.

Чтобы почувствовать, что это такое, рассмотрим таблицу 3.1, где приведены числа, иллюстрирующие скорость роста для нескольких разных функций.

Если считать, что числа в таблице соответствуют микросекундам, то для задачи с *n=1048576* элементами алгоритму с временем работы порядка *O(log n)* потребуется 20 микросекунд, алгоритму со временем порядка *O(n)* – 17 минут, а алгоритму с временем работы порядка $O(n^2)$ – более 12 дней. Теперь преимущество алгоритма 2 с оценкой *O(n)* перед алгоритмом 1 достаточно очевидно.

Таблица 3.1

n	**$\log_2 n$**	**$n \log_2 n$**	**n^2**
1	0	0	1
16	4	64	256
256	8	2 048	65 536
4 096	12	49 152	16 777 216
65 536	16	1 048 565	4 294 967 296
1 048 576	20	20 969 520	1 099 301 922 576
16 775 616	24	402 614 784	281 421 292 179 456

Наилучшей является оценка *O(1)*. В этом случае время вообще не зависит от *n*, т.е постоянно при любом количестве элементов.

Таким образом, *O( )* – "урезанная" верхняя оценка временной сложности алгоритма, отражающая поведение этой сложности в пределе при увеличении размера задачи до бесконечности. Она называется *асимптотической временной сложностью* или *верхним порядком роста временной сложности.*

Приведем математическое толкование символа *O( )*.

Определение 3.1. *O(f(n))* – множество функций *T(n)*, для которых существуют такие константы *c* и n_0, что $T(n) \le c\, f(n)$ для всех $n \ge n_0$. Запись *T(n) = O(f(n))* дословно обозначает, что *T(n)* принадлежит множеству *O(f(n))*. Обратное выражение *O(f(n)) = T(n)* не имеет смысла.

В частности, можно сказать, что $T(n) = 50\, n$ принадлежит $O(n^2)$. Здесь мы имеем дело с неточной оценкой. Разумеется, $T(n) \le 50\, n^2$ при $n \ge 1$, однако

более сильным утверждением было бы $T(n) = O(n)$, так как для $c=50$ и $n_0 = 1$ верно $T(n) \le cn, n \ge n_0$.

Наряду с оценкой $O(n)$ используется оценка $\Omega(n)$ [читается как "*Омега большое от эн*"]. Она обозначает нижнюю оценку роста функции.

Определение 3.2. $\Omega(g(n))$ – множество функций $T(n)$, для которых существуют такие константы c и n_0, что $c\ g(n) \le T(n)$ для всех $n \ge n_0$. В этом случае говорят, что $T(n)$ имеет нижний порядок роста $g(n)$.

Например, пусть количество операций алгоритма описывает функция $T(n) = \Omega(n^2)$. Это значит, что даже в самом удачном случае будет произведено не менее порядка n^2 действий. В то время как оценка $T(n) = O(n^3)$ гарантирует, что в самом худшем случае действий будет порядка n^3, не больше.

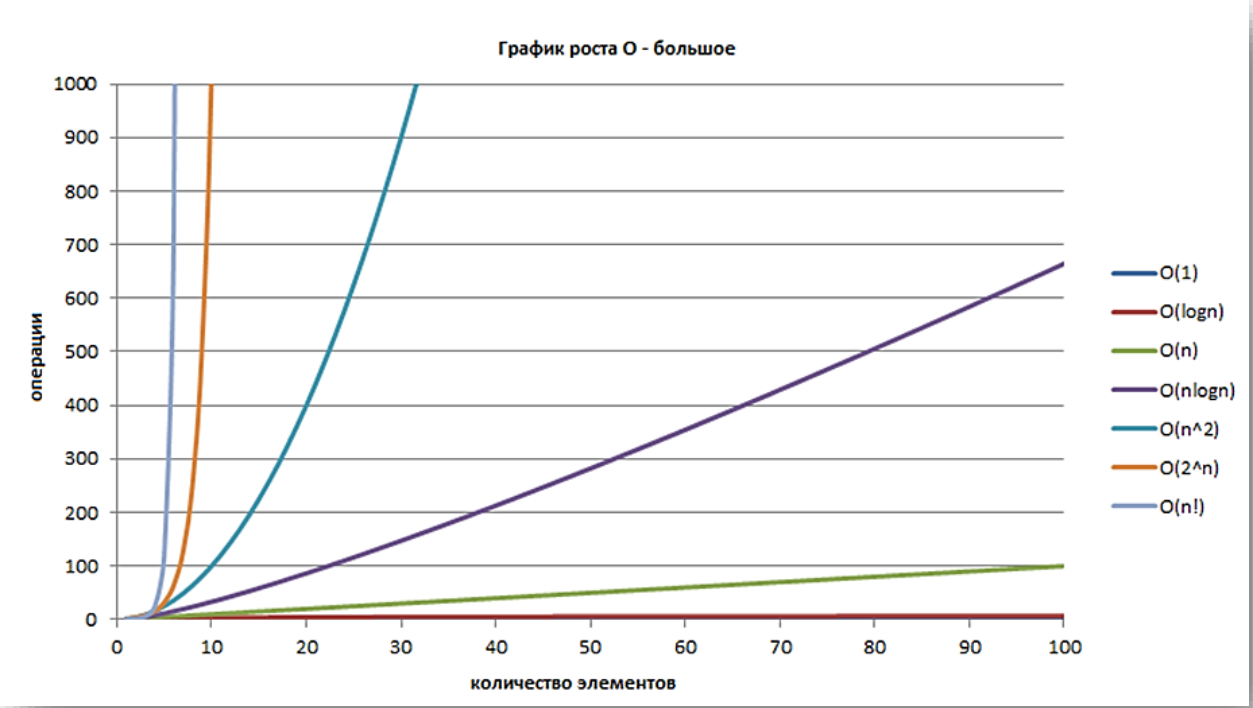
Также используется оценка $\Theta(n)$ [читается как "*Тэта от эн*"], которая является комбинаций $O(n)$ и $\Omega(n)$. Она является точной оценкой асимптотики.

Определение 3.3. $\Theta(g(n))$ – множество функций $T(n)$, для которых существуют такие константы c_1, c_2 и n_0, что $c_1\ g(n) \le T(n) \le c_2\ g(n)$ для всех $n \ge n_0$. Оценка $\Theta(g(n))$ существует только тогда, когда $O(g(n))$ и $\Omega(g(n))$ совпадают и равна им.

Для рассмотренных выше алгоритмов вычисления многочлена временная сложность первого алгоритма порядка $O(n^2)$, $\Omega(n^2)$ и $\Theta(n^2)$, а второго – порядка $O(n)$, $\Omega(n)$ и $\Theta(n)$. Если добавить к первому алгоритму проверки на x=0 в возведении в степень, то на самых удачных исходных данных (когда x=0) имеем порядка n проверок, 0 умножений и 1 сложение, что дает новую оценку $\Omega(n)$ вместе со старой $O(n^2)$. Как правило, основное внимание все же обращается на верхнюю оценку $O(\ )$, поэтому, несмотря на "улучшение", алгоритм 2 остается предпочтительнее.

Итак, $O(\ )$ – асимптотическая оценка алгоритма на худших входных данных, $\Omega(\ )$ – на лучших входных данных, $\Theta(\ )$ – сокращенная запись одинаковых $O(\ )$ и $\Omega(\ )$. Интуитивный смысл этих оценок:

$$При\ n \to \infty\ T(n) = \begin{cases} O(g(n)) : T(n)\ растет\ не\ быстрее\ g(n); \\ \Omega(g(n)) : T(n)\ растет\ не\ медленне\ g(n); \\ \Theta(g(n)) : T(n)\ растет\ так\ же,\ как\ и\ g(n). \end{cases} \quad (3.9)$$



3.4. Вычисление временной сложности

Теоретическое нахождение времени выполнения программ (даже без определения констант пропорциональности) - сложная математическая задача. Однако на практике определение времени выполнения (также без нахождения значения констант) является вполне разрешимой задачей – для этого нужно знать только несколько базовых принципов.

1. *При оценке за функцию берется количество операций, возрастающее быстрее всего.* То есть, если в программе одна функция, например, умножение, выполняется $O(n)$ раз, а сложение – $O(n^2)$ раз, то общая сложность программы – $O(n^2)$, так как в конце концов при увеличении n более быстрые (в определенное, константное число раз) сложения станут выполняться настолько часто, что будут влиять на быстродействие куда

больше, нежели медленные, но редкие умножения. Это принцип обуславливает *правило сумм асимптотических соотношений*. Пусть $T_1(n)$ и $T_2(n)$ — время выполнения двух программных фрагментов P_1 и P_2, $T_1(n)$ имеет степень роста $O(f(n))$, а $T_2(n)$ — $O(g(n))$. Тогда $T_1(n) + T_2(n)$, т.е. время последовательного выполнения фрагментов P_1 и P_2, имеет степень роста $O(max(f(n), g(n)))$.

2. *При оценке O( ) константы не учитываются*. Пусть один алгоритм делает $2500n + 1000$ операций, а другой – $2n+1$. Оба они имеют оценку $O(n)$, так как их время выполнения растет линейно. Этот принцип является следствием *правила произведений*, которое заключается в следующем. Если $T_1(n)$ и $T_2(n)$ имеют степени роста $O(f(n))$ и $O(g(n))$ соответственно, то произведение $T_1(n) \cdot T_2(n)$ имеет степень роста $O(f(n)g(n))$.

3. Другое следствие опускания константы – алгоритм со временем $O(n^2)$ может работать значительно быстрее алгоритма $O(n)$ при малых n. За счет того, что реальное количество операций первого алгоритма может быть $n^2 + 10n + 6$, а второго – $1000000n + 5$. Впрочем, второй алгоритм рано или поздно обгонит первый. n^2 растет куда быстрее $1000000n$.

4. *Основание логарифма внутри символа O( ) не пишется*. Причина этого весьма проста. Пусть у нас есть $O(\log_2 n)$. Но $\log_2 n = \log_3 n / \log_3 2$, а $\log_3 2$, как и любую константу, асимптотика – символ $O( )$ не учитывает. Таким образом, $O(\log_2 n) = O(\log_3 n)$. К любому основанию мы можем перейти аналогично, а значит, и писать его не имеет смысла.

Теперь дадим несколько правил анализа программ. В общем случае время выполнения оператора или группы операторов можно параметризовать с помощью размера входных данных и/или одной или нескольких переменных. Но для времени выполнения программы в целом допустимым параметром может быть только n, размер входных данных.

1. Время выполнения оператора присваивания складывается из времени вычисления выражения и времени выполнения операции присваивания. Если выражение не содержит вызовов функций (в том числе перегруженных операций), то на его вычисление тратится некоторое постоянное время. Операция присваивания так же требует некоторого постоянного времени.

Таким образом, время выполнения операторов присваивания обычно имеет порядок *O(1).*

2. Время выполнения операторов ввода – вывода обычно имеет порядок *O(1).*

3. Время выполнения последовательности операторов определяется с помощью правила сумм. Поэтому степень роста времени выполнения последовательности операторов без определения констант пропорциональности совпадает с наибольшим временем выполнения оператора в данной последовательности.

4. Время выполнения условных операторов состоит из времени выполнения условно исполняемых операторов и времени вычисления самого логического выражения. Время вычисления логического выражения обычно имеет порядок *O(1).* Время для всей конструкции ***if-then-else*** состоит из времени вычисления логического выражения и наибольшего из времени, необходимого для выполнения операторов, исполняемых при значении логического выражения ***true*** (истина) и при значении ***false*** (ложь).

5. Время выполнения цикла является суммой времени всех исполняемых итераций цикла, в свою очередь состоящих из времени выполнения операторов тела цикла и времени вычисления условия прекращения цикла (обычно последнее имеет порядок *O(1)*). Часто время выполнения цикла вычисляется, пренебрегая определением констант пропорциональности, как произведение количества выполненных итераций цикла на наибольшее возможное время выполнения операторов тела цикла. Время выполнения каждого цикла, если в программе их несколько, должно определяться отдельно.

6. Для программ, содержащих несколько процедур или функций (среди которых нет рекурсивных), можно подсчитать общее время выполнения программы путем последовательного нахождения времени выполнения этих подпрограмм, начиная с той, которая не имеет вызовов других процедур или функций. Так как мы предположили, что все подпрограммы нерекурсивные, то должна существовать хотя бы одна подпрограмма, не имеющая вызовов других процедур или функций. Затем можно определить время выполнения

подпрограмм, вызывающих эту подпрограмму, используя уже вычисленное время выполнения вызываемой подпрограммы. Продолжая этот процесс, найдем время выполнения всех подпрограмм и, наконец, время выполнения всей программы.

7. Если есть рекурсивные процедуры или функции, то нельзя упорядочить все подпрограммы таким образом, чтобы каждая подпрограмма вызывала только процедуры или функции, время выполнения которых подсчитано на предыдущем шаге. В этом случае мы должны с каждой рекурсивной процедурой (функцией) связать временною функцию *T(n),* где *n* определяет объем ее аргументов. Затем мы должны получить *рекуррентное соотношение* для *T(n),* т.е. уравнение (или неравенство) для *T(n),* где участвуют значения *T(k)* для различных значений $k<n$.

Общий метод решения таких рекуррентных соотношений состоит в последовательном раскрытии выражений *T(к)* в правой части уравнения (путем подстановки в исходное соотношение *k* вместо *n*) до тех пор, пока не получится формула, у которой в правой части отсутствует *T*. При этом часто приходится находить суммы различных последовательностей; если значения таких сумм нельзя вычислить точно, то для сумм находятся верхние границы, что позволяет, в свою очередь, получить верхние границы для *T(n)*.

Вопросы для повторения темы 3

1. Каковы критерии выбора алгоритма?
2. Какие факторы оказывают влияние на время выполнения программы?
3. Что называют временной сложностью алгоритма?
4. Что такое верхний и нижний порядки роста временной сложности?
5. Перечислите базовые принципы, применяемые при определении значения асимптотической временной сложности.
6. Какие правила анализа алгоритмов на предмет оценки временной сложности вы знаете?
7. Какова временная сложность тривиального алгоритма вычисления значения многочлена? А алгоритма вычисления значения многочлена по схеме Горнера?